

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ  
Федеральное государственное автономное  
образовательное учреждение высшего образования  
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

**Департамент цифровых, робототехнических систем и электроники**

**«Условные операторы и циклы в языке Python»  
Отчет по лабораторной работе № 3  
по дисциплине «Программирование на Python»  
Вариант 9**

Выполнил студент группы ИВТ-б-о-24-1  
Каиров Вадим Сосланович  
«\_\_» октября 2025г.

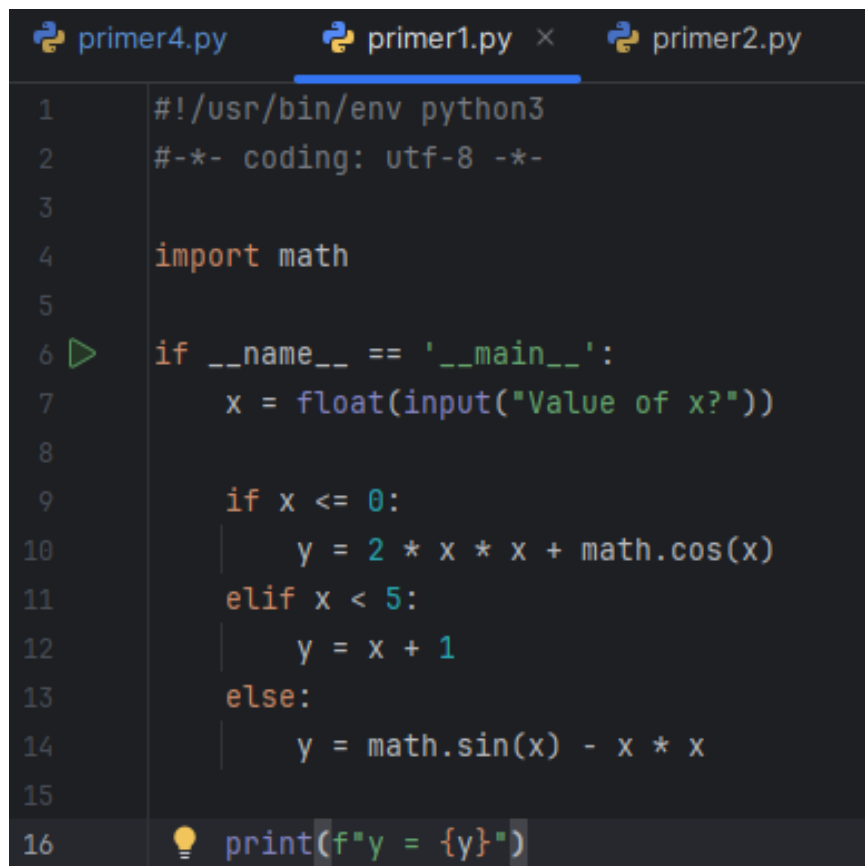
Подпись студента \_\_\_\_\_  
Работа защищена « » \_\_\_\_\_ 20\_\_ г.  
Проверил Воронкин Р.А. \_\_\_\_\_  
(подпись)

**Цель работы:** приобретение навыков программирования разветвляющихся алгоритмов и алгоритмов циклической структуры. Освоить операторы языка Python версии 3.x if, while, for, break и continue, позволяющих реализовывать разветвляющиеся алгоритмы и алгоритмы циклической структуры.

**Ход работы:**

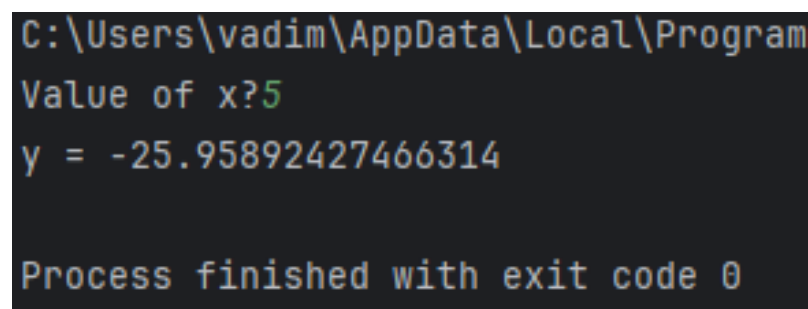
Ссылка на GitHub: <https://github.com/ozetin/labochka3>

В PyCharm были прописаны и разобраны все примеры из методического материала.



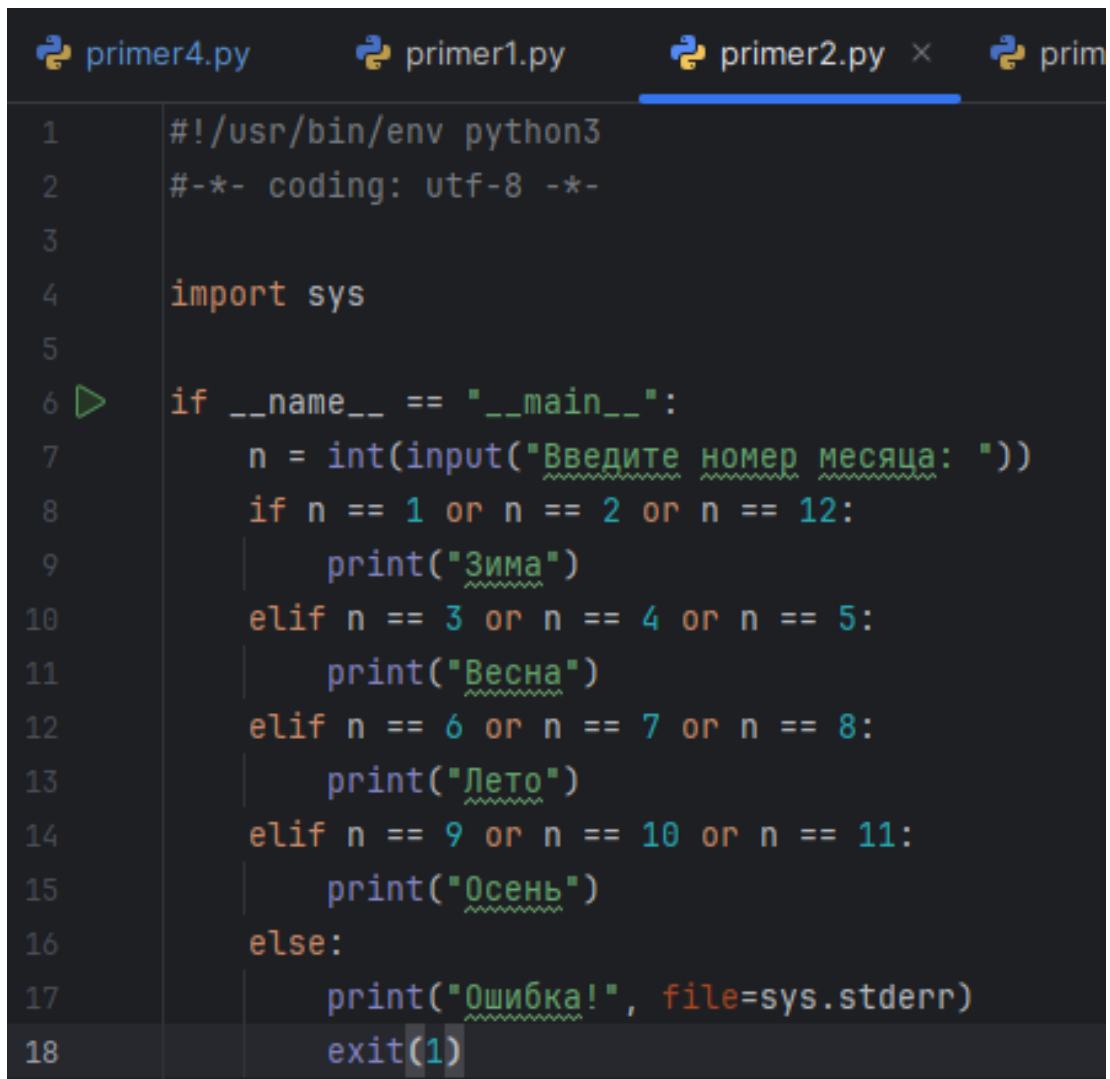
```
1 #!/usr/bin/env python3
2 -*- coding: utf-8 -*-
3
4 import math
5
6 if __name__ == '__main__':
7     x = float(input("Value of x?"))
8
9     if x <= 0:
10         y = 2 * x * x + math.cos(x)
11     elif x < 5:
12         y = x + 1
13     else:
14         y = math.sin(x) - x * x
15
16     print(f'y = {y}')
```

Рисунок 1. Пример 1 лабораторной работы



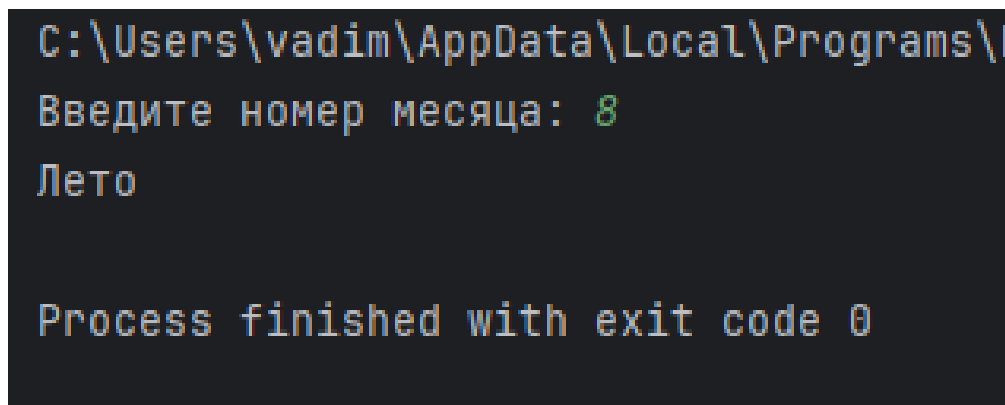
```
C:\Users\vadim\AppData\Local\Program
Value of x?5
y = -25.95892427466314
Process finished with exit code 0
```

Рисунок 2. Результат выполнения примера 1



```
primer4.py primer1.py primer2.py × prim
1  #!/usr/bin/env python3
2  -*- coding: utf-8 -*-
3
4  import sys
5
6  if __name__ == "__main__":
7      n = int(input("Введите номер месяца: "))
8      if n == 1 or n == 2 or n == 12:
9          print("Зима")
10     elif n == 3 or n == 4 or n == 5:
11         print("Весна")
12     elif n == 6 or n == 7 or n == 8:
13         print("Лето")
14     elif n == 9 or n == 10 or n == 11:
15         print("Осень")
16     else:
17         print("Ошибка!", file=sys.stderr)
18     exit(1)
```

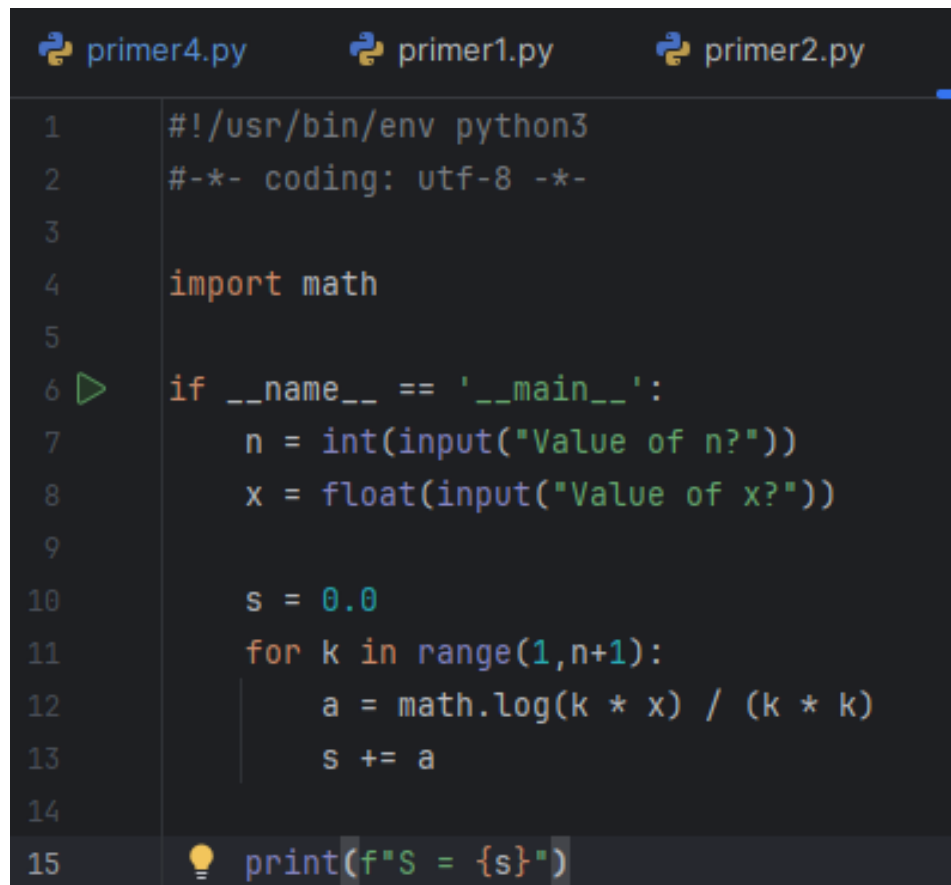
Рисунок 3. Пример 2 лабораторной работы



```
C:\Users\vadim\AppData\Local\Programs\
Введите номер месяца: 8
Лето

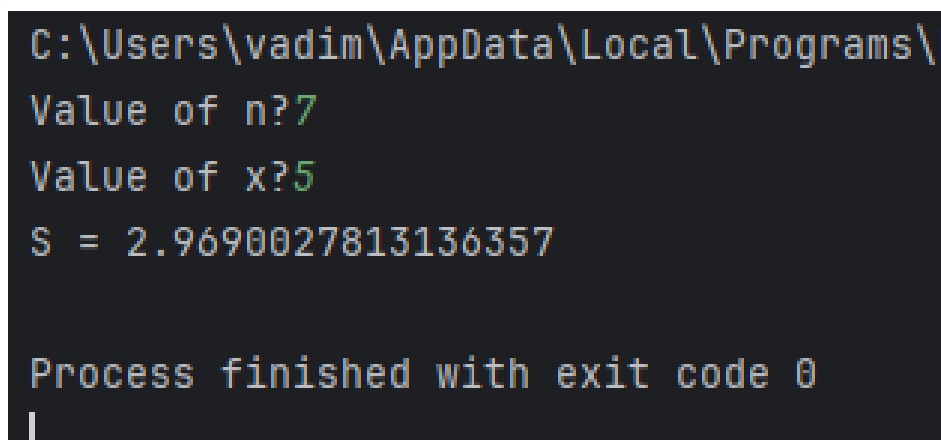
Process finished with exit code 0
```

Рисунок 4. Результат выполнения примера 2



```
primer4.py    primer1.py    primer2.py
1  #!/usr/bin/env python3
2  #-*- coding: utf-8 -*-
3
4  import math
5
6  if __name__ == '__main__':
7      n = int(input("Value of n?"))
8      x = float(input("Value of x?"))
9
10     s = 0.0
11     for k in range(1,n+1):
12         a = math.log(k * x) / (k * k)
13         s += a
14
15     print(f"S = {s}")
```

Рисунок 5. Пример 3 лабораторной работы



```
C:\Users\vadim\AppData\Local\Programs\
Value of n?7
Value of x?5
S = 2.9690027813136357

Process finished with exit code 0
```

Рисунок 6. Результат выполнения примера 3

Для следующих примеров необходимо было создать UML-диаграммы деятельности

```

primer4.py x primer1.py primer2.py primer3.py
1  #!/usr/bin/env python3
2  -*- coding: utf-8 -*-
3
4  import math
5  import sys
6
7  if __name__ == '__main__':
8      a = float(input("Value of a?"))
9      if a < 0:
10         print("illegal value of a", file=sys.stderr)
11         exit(1)
12
13     x, eps = 1, 1e-10
14     while True:
15         xp = x
16         x = (x + a / x) / 2
17         if math.fabs(x - xp) < eps:
18             break
19
20     print(f"x = {x}\nX = {math.sqrt(a)}")

```

Рисунок 7. Пример 4 лабораторной работы

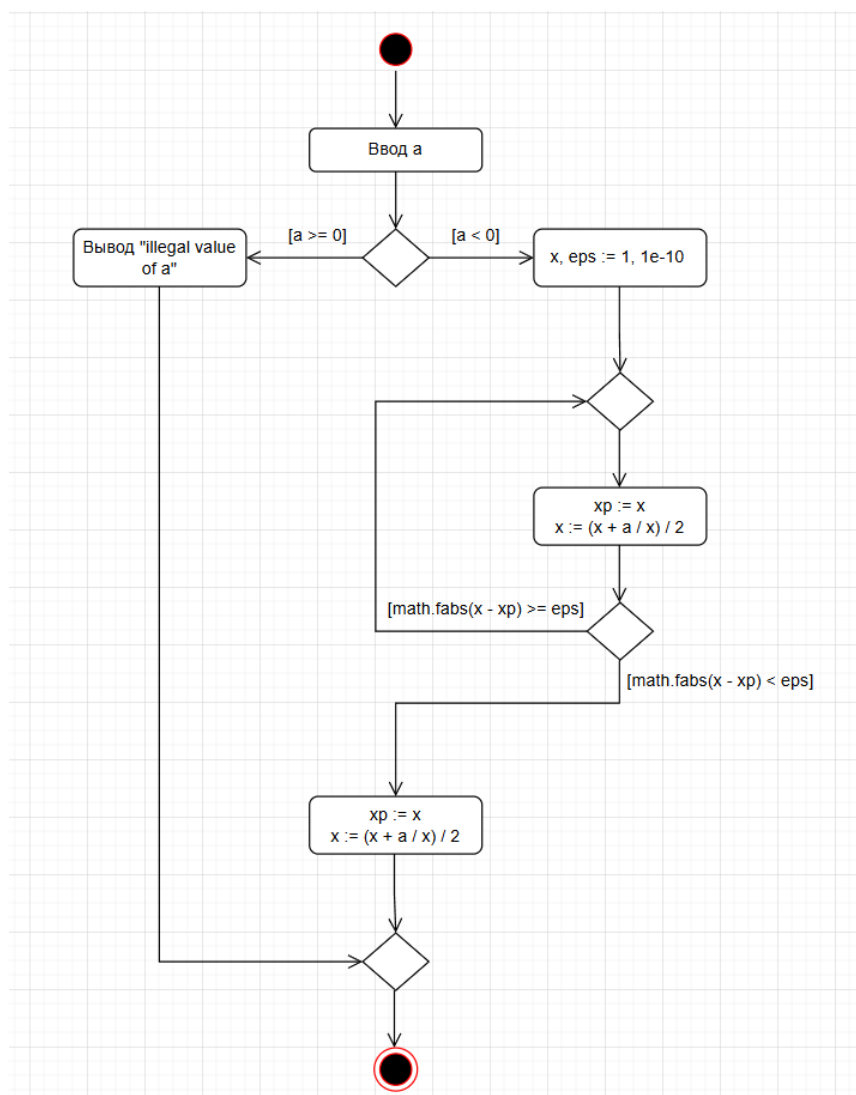
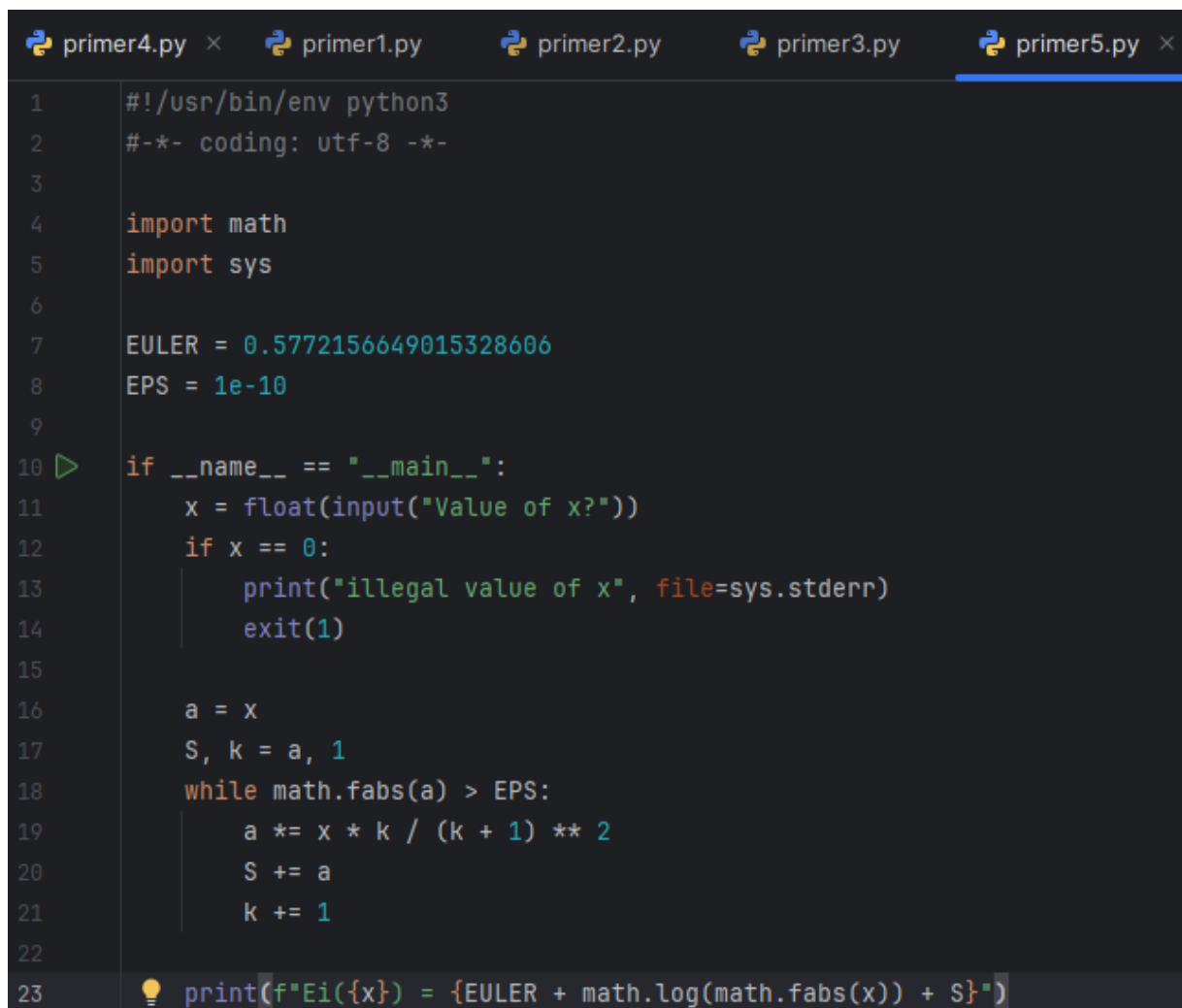


Рисунок 8. UML-диаграмма деятельности примера 4

```
C:\Users\vadim\AppData\Local\Programs\
Value of a?7
x = 2.6457513110645907
X = 2.6457513110645907

Process finished with exit code 0
```

Рисунок 9. Результат выполнения примера 4



```
primer4.py × primer1.py primer2.py primer3.py primer5.py ×
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  import math
5  import sys
6
7  EULER = 0.5772156649015328606
8  EPS = 1e-10
9
10 if __name__ == "__main__":
11     x = float(input("Value of x?"))
12     if x == 0:
13         print("illegal value of x", file=sys.stderr)
14         exit(1)
15
16     a = x
17     S, k = a, 1
18     while math.fabs(a) > EPS:
19         a *= x * k / (k + 1) ** 2
20         S += a
21         k += 1
22
23     print(f"Ei({x}) = {EULER + math.log(math.fabs(x)) + S}")
```

Рисунок 10. Пример 5 лабораторной работы

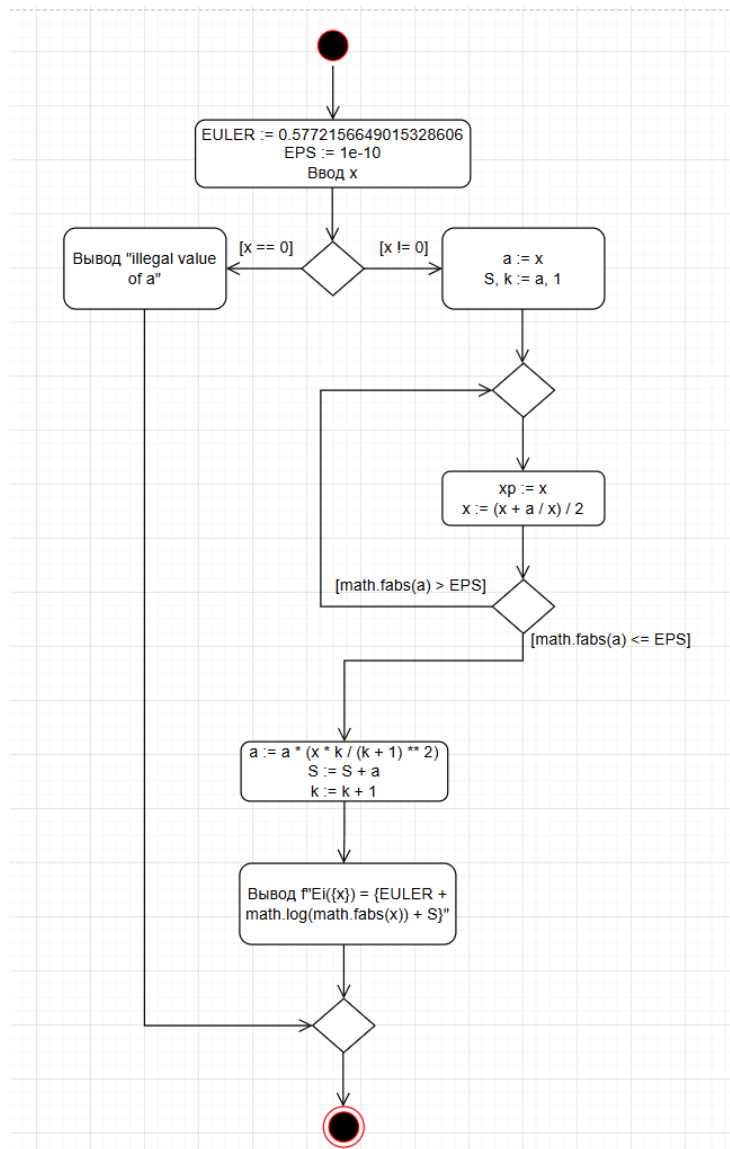


Рисунок 11. UML-диаграмма деятельности примера 5

```

C:\Users\vadim\AppData\Local\Programs
Value of x?10
Ei(10.0) = 2492.228976241855

Process finished with exit code 0

```

Рисунок 12. Результат выполнения примера 5

Далее были выполнены задания и по ним также были сделаны UML-диаграммы.

Задание 1. Вводится число экзаменов  $N \leq 20$ . Напечатать фразу Мы успешно сдали  $N$  экзаменов, согласовав слово “экзамен” с числом  $N$ .

```
zadanie1.py x zadanie4.py zadanie2.py zadanie3.py
1  #!/usr/bin/env python3
2  -*- coding: utf-8 -*-
3
4  > if __name__ == '__main__':
5      N = int(input("Введите число экзаменов: "))
6
7      if N == 1:
8          print(f"Мы успешно сдали {N} экзамен")
9
10     elif 1 < N < 5:
11         print(f"Мы успешно сдали {N} экзамена")
12
13     elif 5 <= N <= 20:
14         print(f"Мы успешно сдали {N} экзаменов")
15
16     else:
17         print("Нужно ввести число от 1 до 20")
```

Рисунок 13. Решение задания 1

```
Введите число экзаменов: 10
Мы успешно сдали 10 экзаменов
```

Рисунок 14. Результат выполнения задания 1

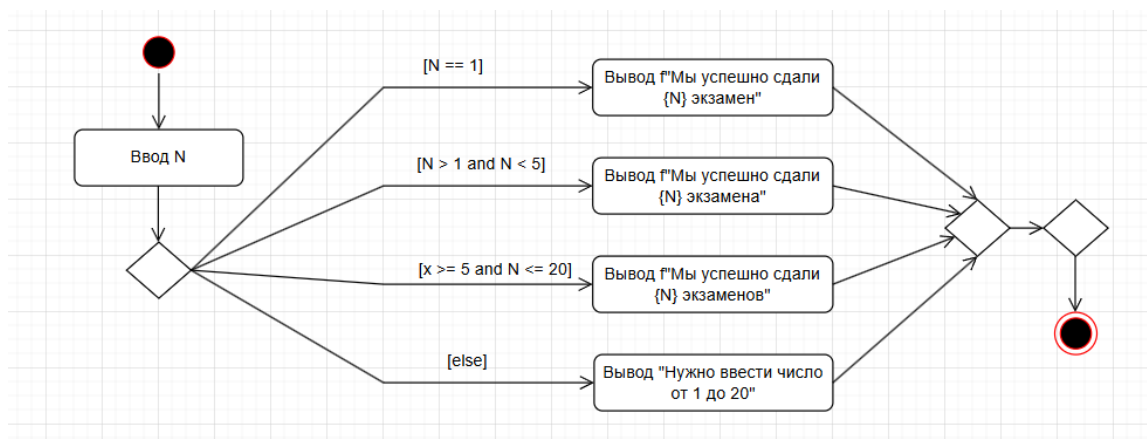


Рисунок 15. UML-диаграмма деятельности задания 1

Задание 2. Найти координаты точки пересечения прямых, заданных уравнениями  $a_1x + b_1y + c_1 = 0$  и  $a_2x + b_2y + c_2 = 0$ , либо сообщить, совпадают, параллельны или не существуют.



```

1  #!/usr/bin/env python3
2  -*- coding: utf-8 -*-
3
4  if __name__ == '__main__':
5      a1 = int(input("Введите значение a1: "))
6      b1 = int(input("Введите значение b1: "))
7      c1 = int(input("Введите значение c1: "))
8
9      a2 = int(input("Введите значение a2: "))
10     b2 = int(input("Введите значение b2: "))
11     c2 = int(input("Введите значение c2: "))
12
13     if a1 == a2 and b1 == b2 and c1 == c2:
14         print("Прямые совпадают")
15
16     elif a1 == 0 and b1 == 0 and c1 != 0:
17         print("Прямая 1 не существует")
18
19     elif a2 == 0 and b2 == 0 and c2 != 0:
20         print("Прямая 2 не существует")
21
22     elif a1 == a2 and b1 == b2:
23         print("Прямые параллельны")
24
25     else:
26         k1 = -a1 / b1
27         b1_new = -c1 / b1
28
29         k2 = -a2 / b2
30         b2_new = -c2 / b2
31
32         x = (b2_new - b1_new) / (k1 - k2)
33         y = k1 * x + b1_new
34
35     print(f"Точка пересечения: x = {x}, y = {y}")

```

Рисунок 16. Решение задания 2

```

C:\Users\vadim\AppData\Local\Programs\Python
Введите значение a1: 1
Введите значение b1: 6
Введите значение c1: 4
Введите значение a2: 8
Введите значение b2: 5
Введите значение c2: 3
0.04651162790697674 -0.6744186046511628

```

Рисунок 17. Результат выполнения задания 2

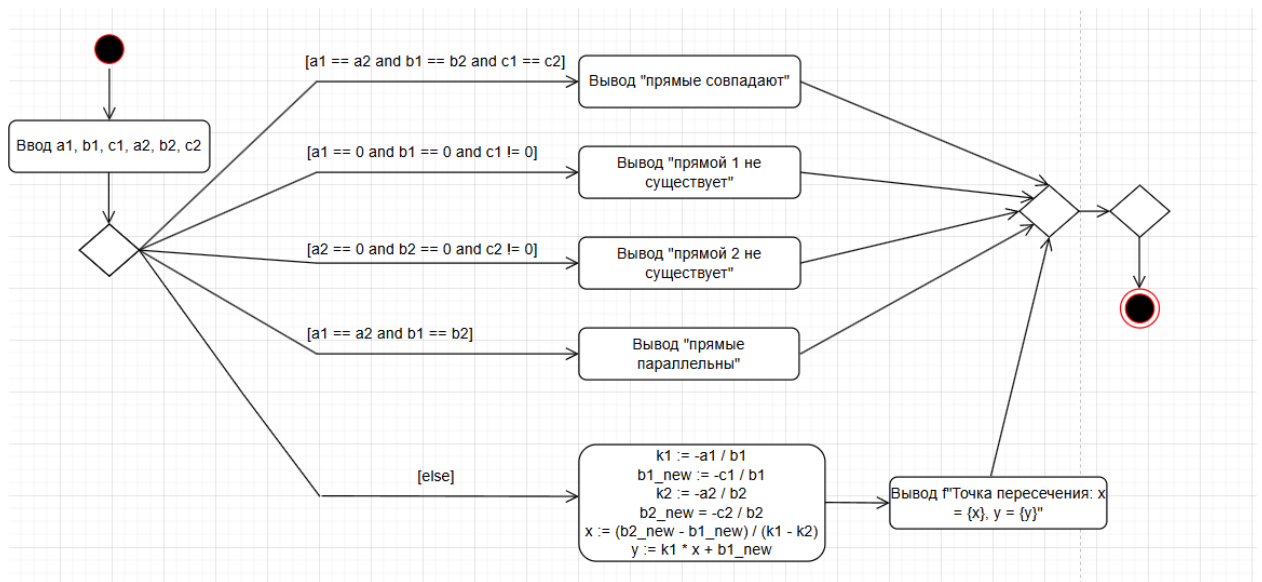


Рисунок 18. UML-диаграмма деятельности задания 2

Задание 3. Если к сумме цифр двузначного числа прибавить квадрат этой суммы, то снова получится это двузначное число. Найти все эти числа.

```

1  #!/usr/bin/env python3
2  -*- coding: utf-8 -*-
3
4  if __name__ == '__main__':
5      for i in range(10, 100):
6          summ = i % 10 + i // 10
7          summ += summ ** 2
8
9          if summ == i:
10             print(i)
  
```

Рисунок 19. Решение задания 3

```

C:\Users\vadim\AppData\Local\Programs
12
42
90

Process finished with exit code 0
  
```

Рисунок 20. Результат выполнения задания 3

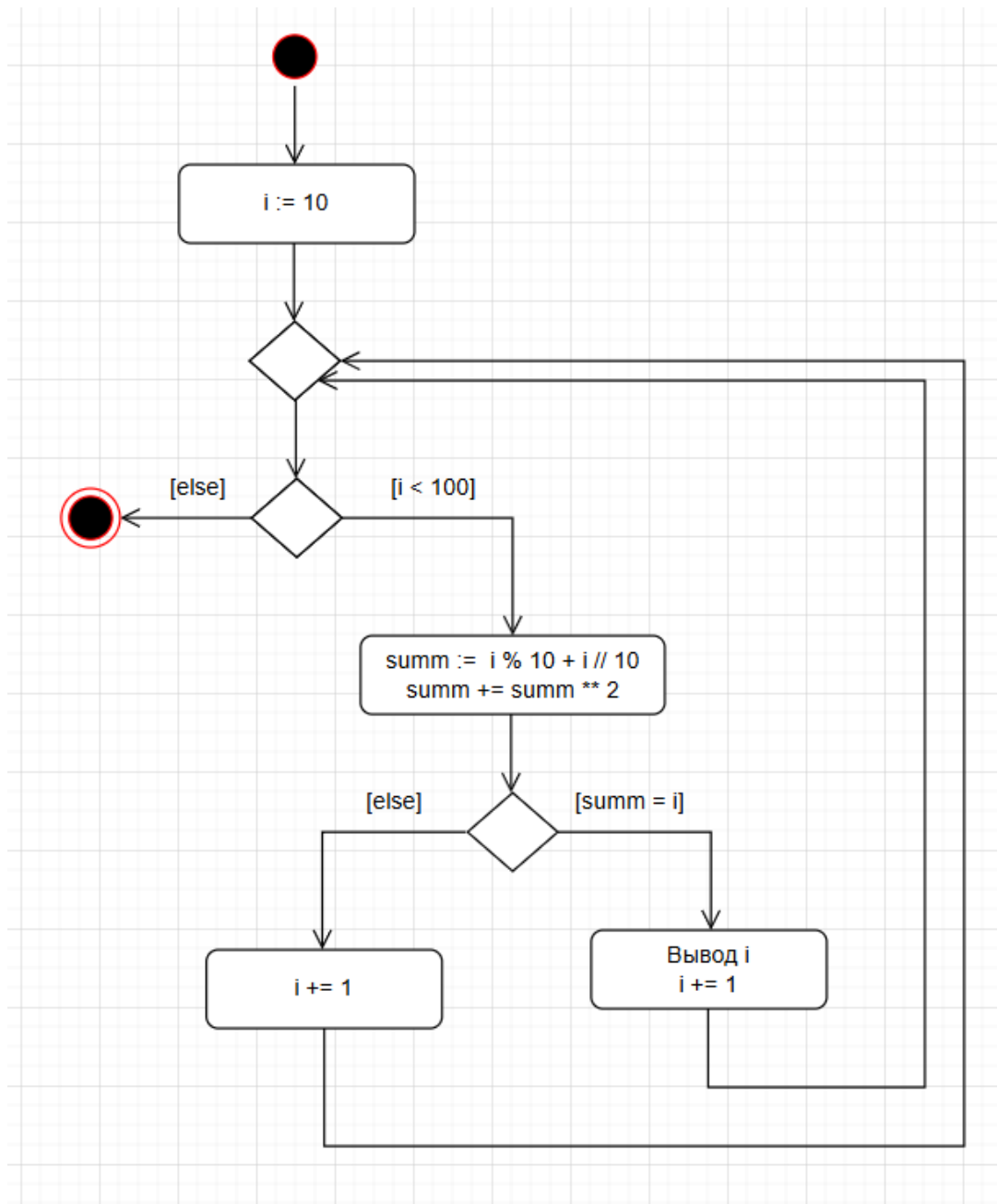


Рисунок 21. UML-диаграмма деятельности задания 3

Задание 4. Составить UML-диаграмму деятельности, программу и произвести вычисления вычисление значения дилогарифма по ее разложению в ряд с точностью  $\varepsilon = 10^{-10}$ , аргумент дилогарифма  $x$  вводится с клавиатуры.

```

1  #!/usr/bin/env python3
2  #-*- coding: utf-8 -*-
3
4  import sys
5
6  EPS = 1e-10
7
8  if __name__ == '__main__':
9      x = float(input("value of x?"))
10     a = x
11     S, k = a, 1
12     if not (0 <= x <= 2):
13         print("illegal value of x", file=sys.stderr)
14     else:
15         S = 0.0
16         k = 1
17         a = 1.0
18
19     while abs(a) > EPS:
20         a = ((-1) ** k) * (x - 1) ** k / (k ** 2)
21         S += a
22         k += 1
23
24     print(f"f({x}) = {S}")

```

Рисунок 22. Решение задания повышенной сложности

```

C:\Users\vadim\AppData\Local\Programs
value of x?1.5
f(1.5) = -0.4484142069387109

Process finished with exit code 0

```

Рисунок 23. Вывод решения задания повышенной сложности

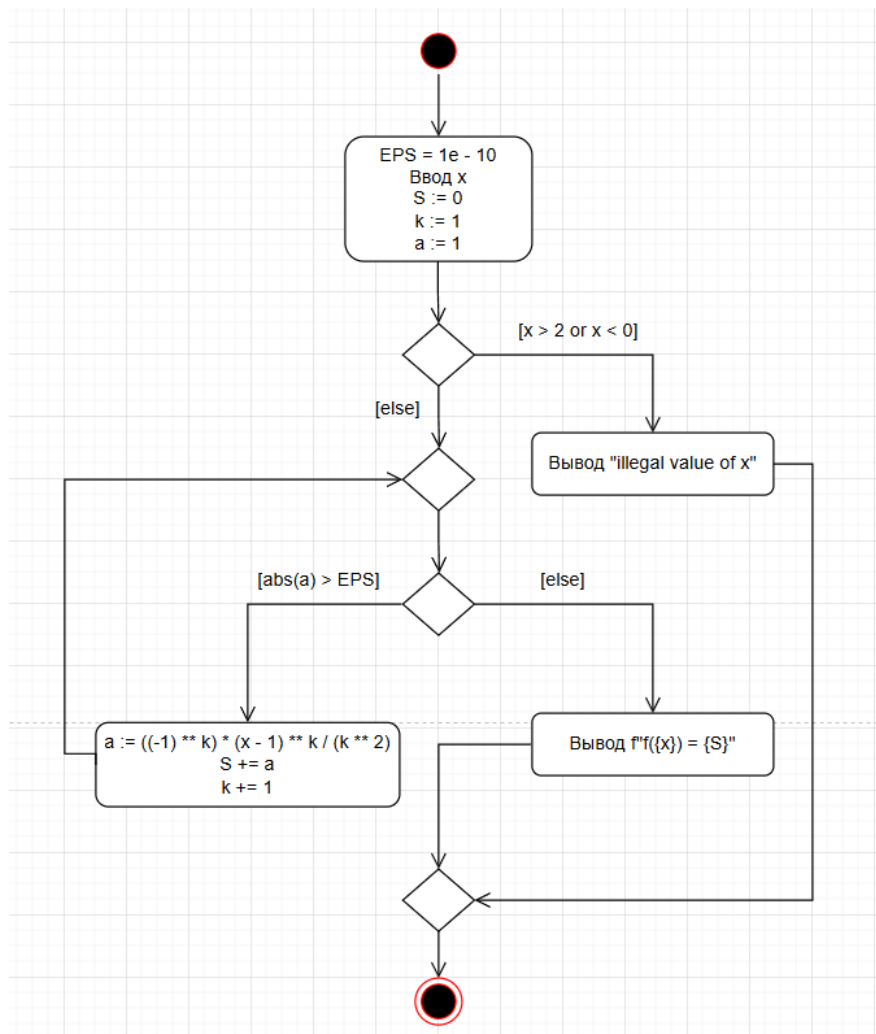


Рисунок 24. UML-диаграмма решения задания повышенной сложности

### Контрольные вопросы:

1) Для чего нужны диаграммы деятельности UML?

Диаграммы деятельности в UML нужны для описания логики поведения системы, т. е. что и в каком порядке происходит при выполнении процессов

2) Что такое состояние действия и состояние деятельности?

Состояние действия – это элементарная операция, выполняемая быстро и целиком, без внутренней структуры. То есть одно конкретное действие, которое нельзя разбить на подэтапы внутри этой диаграммы.

Состояние деятельности — это комплексный процесс, который может включать несколько действий, ветвления, циклы или даже отдельную вложенную диаграмму деятельности.

3) Какие нотации существуют для обозначения переходов и ветвлений в диаграммах деятельности?

1. Переход
2. Ветвление
3. Слияние

4) Какой алгоритм является алгоритмом разветвляющейся структуры?

Алгоритм разветвляющейся структуры — это алгоритм, в котором ход выполнения зависит от условия. Иными словами, в нём есть выбор между несколькими вариантами действий (веток) — в зависимости от результата проверки какого-либо логического выражения.

5) Чем отличается разветвляющийся алгоритм от линейного?

Линейный алгоритм выполняет действия последовательно, одно за другим, без каких-либо проверок и выборов. Все шаги выполняются в строгом порядке сверху вниз. Каждый шаг выполняется всегда, независимо от входных данных.

6) Что такое условный оператор? Какие существуют его формы?

Условный оператор — это конструкция, которая позволяет программе выполнить разные действия в зависимости от выполнения логического условия.

1. Полная (if ... else)
2. Неполная (if без else)
3. Множественная (if ... elif ... else)

7) Какие операторы сравнения используются в Python?

==, !=, >, <, >=, <=

8) Что называется простым условием? Приведите примеры

Простое условие — это логическое выражение, в котором сравниваются два значения с помощью одного оператора сравнения. Пример:  $X > 10$

9) Что такое составные условия? Приведите примеры

Если простое условие — это одна проверка, то составное условие объединяет несколько таких проверок, чтобы программа могла принимать более сложные решения. Пример:  $X > 5$  and  $X < 10$

10) Какие логические операторы допускаются при составлении сложных условий?

And, or, not

11) Может ли оператор ветвления содержать внутри себя другие ветвления?

Да

12) Какой алгоритм является алгоритмом циклической структуры?

Алгоритм циклической структуры — это такой алгоритм, в котором одни и те же действия выполняются несколько раз подряд до выполнения определённого условия.

13) Типы циклов в языке Python.

While – цикл с предусловием

For – цикл перебора

14) Назовите назначение и способы применения функции range.

Функция range() создаёт последовательность целых чисел в заданном диапазоне — от начального значения до конечного (не включая его).

15) Как с помощью функции range организовать перебор значений от 15 до 0 с шагом 2?

```
for i in range(15, -1, -2)
```

16) Могут ли быть циклы вложенными?

Да

17) Как образуется бесконечный цикл и как выйти из него?

Например

```
X = 5
```

```
While x > 3:
```

Выйти с помощью break

18) Для чего нужен оператор break?

Чтобы завершить цикл

19) Где употребляется оператор continue и для чего он используется?

Если `break` полностью останавливает цикл, то `continue` делает более тонкую работу: он пропускает текущую итерацию и переходит к следующей.

20) Для чего нужны стандартные потоки `stdout` и `stderr`?

`Stdout` – Стандартный поток вывода. Используется для нормального вывода программы: сообщений, результатов вычислений

`Stderr` – Стандартный поток ошибок. Предназначен специально для сообщений об ошибках, сбоях или предупреждениях.

21) Как в Python организовать вывод в стандартный поток `stderr`?

```
Print("Ошибка", file=sys.stderr)
```

22) Каково назначение функции `exit`?

Функция `exit()` завершает выполнение программы досрочно, то есть заставляет Python немедленно выйти из программы и вернуть код завершения (число) операционной системе.

Вывод: ходе выполнения работы были приобретены и закреплены навыки использованием разветвляющихся и циклических структур, являющихся основой логики любой программы. Освоены ключевые операторы языка Python 3.x, такие как: `if`, `elif`, `else`, `while`, `for`, `break`, `continue`.